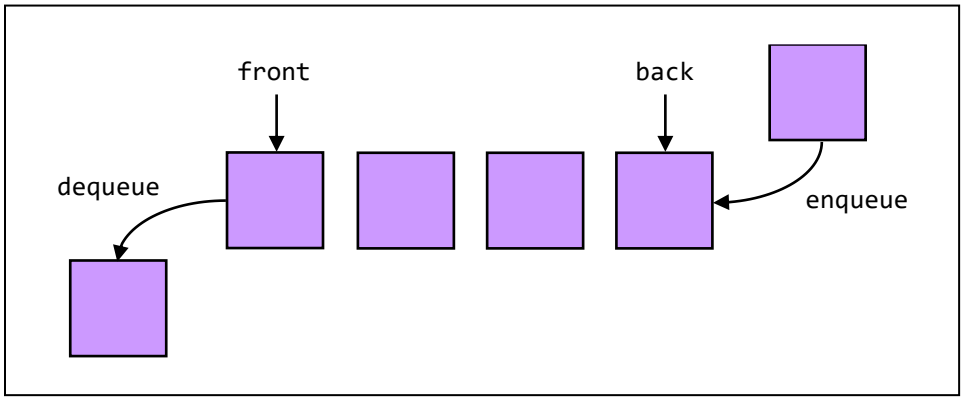
Lab Title: Linked-List Queue Data Structure
Course: CS 1410 – Object Oriented Programming

Objectives: pointers, object-oriented programming, single class program, objects, object instantiation, data members, constructors, member functions, logic.

Overview

In this lab you will use pointers to create a queue data structure in C++. The queue will be implemented using a linked list, in which each node houses a pointer to the next node in the chain. The purpose of the project is to introduce the use of pointers in a practical format.

A queue is a first-in-first-out data structure, in which elements are added to the back and removed from the front (*refer to the diagram below*). Each element in the queue is referred to as a node. The term enqueue is used to describe the process of adding an element to a queue; the term dequeue describes removing an element.

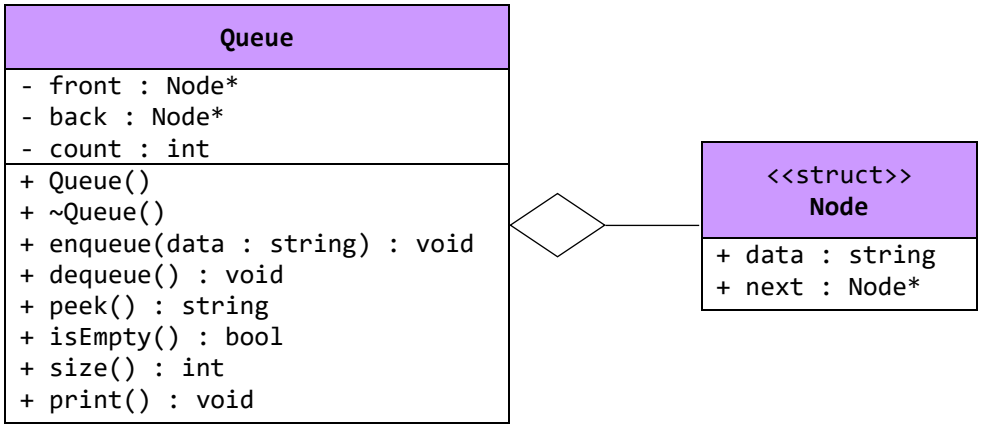


This project is divided into three parts. The first part introduces the `Node` structure and the `Queue` class via a UML diagram. In the second part, each of the member variables and functions are described, along with programming instructions. The third part is used to test the queue to ensure functionality. Complete each section as described.

© Copyright 2023 Lance P. Rhodes, Ph.D. All rights reserved.

Part 1: Node and Queue UML

The following UML diagram depicts the `Node` struct and `Queue` class, both of which will be needed for this project. Code the definitions for the structure and the class, with member variables and functions, as shown. Both may be completed within a single .cpp file. Descriptions of the member variables and functions are provided in Part 2 of this document.



Part 2: Node and Queue Member Descriptions

In this section, you will complete the member variable and function definitions provided by the UML diagram in Part 1. Read each description and complete the logic as described.

`struct: Node`

The `Node` structure contains two member variables. The first public variable, `data` of type `string`, will be used to store the data that is housed in each node. The second public variable, `next`, is a pointer to a `Node` type. This pointer will be used to chain one node in the queue to the next node in the queue.

© Copyright 2023 Lance P. Rhodes, Ph.D. All rights reserved.

`class: Queue`

The `Queue` class contains three private member variables. The `front` and `back` variables are both pointers to a `Node` structure. As a queue adds nodes to the back, and removes nodes from the front, a reference to each location is needed. The integer variable, `count`, will be used to track the number of items in the queue.

`+ Queue()`

The `Queue` default constructor may be used to instantiate a queue object. The constructor should be used to initialize the `front` and `back` pointers to `nullptr`, and the `count` variable to `0`. (Alternatively, these member variables may be initialized upon declaration).

`+ ~Queue()`

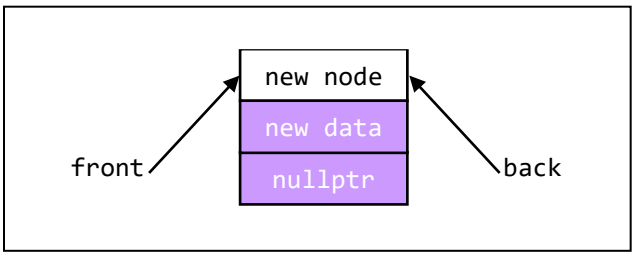
As the `Queue` class will be using the `new` operator to store data on the heap, it will be necessary to manually remove any such references to prevent a memory leak. The `~Queue` destructor is therefore required to systematically delete the data referenced by the node pointers. Fortunately, this may be done by calling the `dequeue()` function—which will be described later in this document—until the queue is empty.

`+ enqueue(data : string)`

The enqueue function is used to add an item to the queue. As a first-in-first-out data (FIFO) data structure, items are added to the back of the line. Complete the steps as provided.

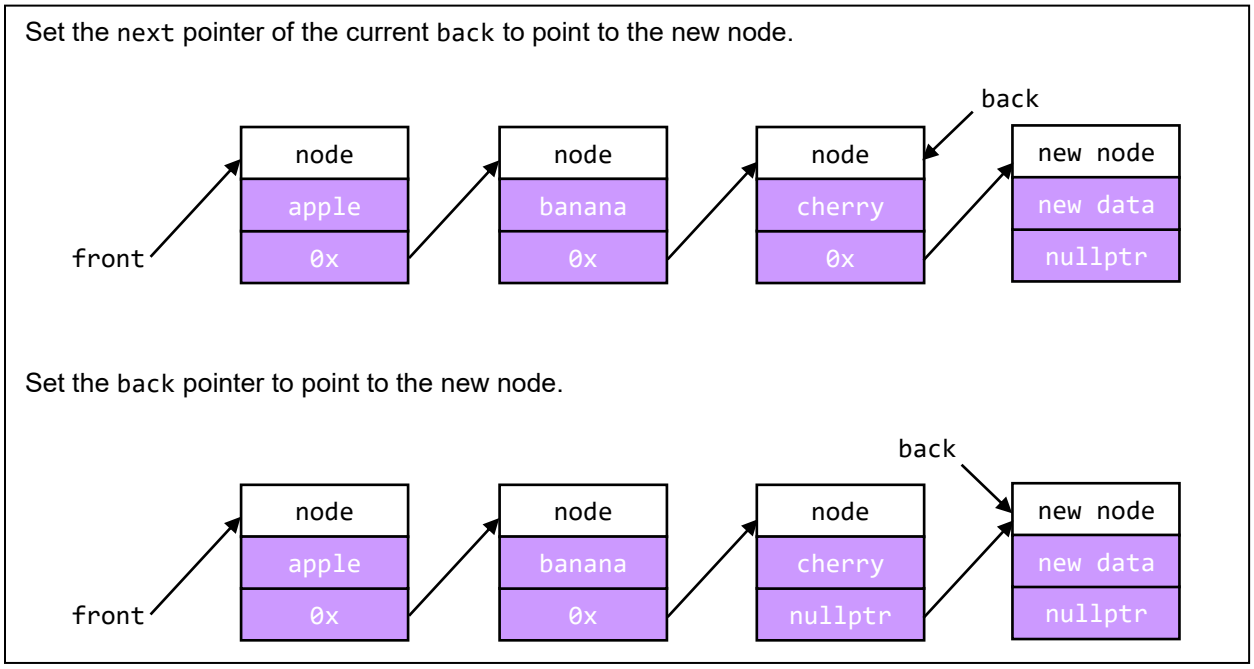
Step 1. Define a pointer which points to a new `Node struct`. The new `Node` should use the passed-in `data` argument for its `data` attribute. Set the `next` data member to `nullptr`, temporarily.

Step 2. If the queue is empty then the new node is both the first and last item in the queue. Set the `front` and `back` pointers to the new node pointer.



© Copyright 2023 Lance P. Rhodes, Ph.D. All rights reserved.

Step 3. If the queue is not empty, the new element will be added to the back of the queue. To do so, set the `next` attribute of the last item (currently represented by the `back` pointer) to the new node pointer. Then set the back pointer to point to the new node pointer (*refer to the diagram below*).



Step 4. Increase the `count` variable by 1.

`dequeue() : void`

The `dequeue()` function is used to remove an item from the queue. As a first-in-first-out (FIFO) data structure, elements are removed from the front of the line. Complete the step as described (*refer to the diagram on the next page*).

Step 1. If the queue is not empty, complete the following:

a. Store the current `front` pointer into a temporary pointer variable. (This is necessary to retain access after the `front` pointer is changed).

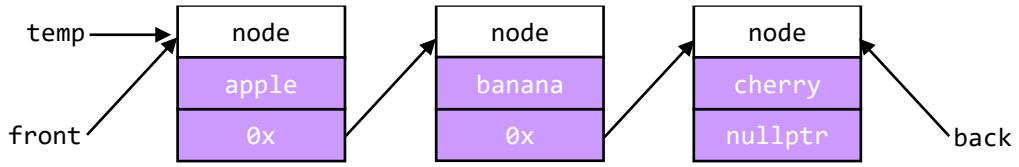
b. Set the `front` pointer to the `next` pointer of the current `front` pointer.

c. Delete the temporary variable.

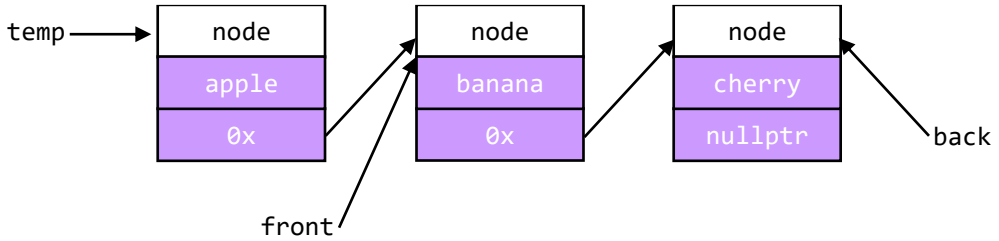
d. Decrease the `count` variable by 1.

© Copyright 2023 Lance P. Rhodes, Ph.D. All rights reserved.

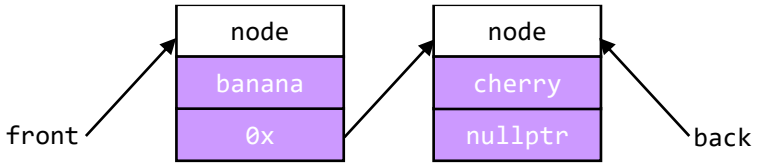
Define a temporary `Node` pointer and assign `front`.



Reassign the `front` pointer to the `next` pointer of the `front` pointer.



Delete the temporary pointer (which points to the old `front` node).



`+ peek() : string`

The `peek()` function is used to retrieve the data from the front of the queue. If the queue is not empty, simply return the `data` member variable, accessed via the `front` pointer. If the queue is empty, return an empty string.

`+ isEmpty() : bool`

`The isEmpty()` function is used to determine if the queue is empty. In this case, the queue is empty if the front pointer is `nullptr`. If the queue is empty, return `true`. Otherwise, return `false`.

`+ size() : int`

The `size()` function returns the number of nodes in the queue.

© Copyright 2023 Lance P. Rhodes, Ph.D. All rights reserved.

```
+ print() : void
```

The `print()` function is used to walk the queue and print each item (in some decorated format). Complete the steps.

Step 1. If the queue is empty, display a message to indicate that the queue is empty.

Step 2. If the queue is not empty, complete the following:

a. Create a temporary `Node` pointer and set it to the `front` pointer.

b. Loop while the temporary pointer is not `nullptr`. Within the loop:

- Display the `data` member variable in some decorated fashion.
- Set the temporary pointer to the `next` pointer of the temporary pointer.

© Copyright 2023 Lance P. Rhodes, Ph.D. All rights reserved.

Part 3: Testing the Queue

In this part of the project, you will complete a variety of tests to ensure the functionality of the `Queue` class.

Add the following test code to the `main` function of your program. In addition to the tests provided, complete at least three more tests of your own. You may add more tests if you wish.

Test code:

```
// Queue starts empty
Queue queue;
queue.print();

// Queue with one node
queue.enqueue("Apple");
cout << queue.size() << " item(s): ";
queue.print();

// Removing one node; queue becomes empty
queue.dequeue();
queue.print();

// Trying to dequeue from empty (no crash)
queue.dequeue();

// Trying to peek empty queue (no crash)
cout << queue.peek();

// Adding three nodes
queue.enqueue("Banana");
queue.enqueue("Cherry");
queue.enqueue("Date");
cout << queue.size() << " item(s): ";
queue.print();

// Removing one node
queue.dequeue();
cout << queue.size() << " item(s): ";
queue.print();

// Adding one additional node
queue.enqueue("Elderberry");
cout << queue.size() << " item(s): ";
queue.print();

// Peeking first node
cout << "Peek: " << queue.peek() << endl;
```

Sample output:

```
The queue is empty.

1 item(s): [Apple]

The queue is empty.

3 item(s): [Banana][Cherry][Date]

2 item(s): [Cherry][Date]

3 item(s): [Cherry][Date][Elderberry]

Peek: Cherry
```

Submit the completed .cpp file online for grading. A grade rubric is provided on the next page.

© Copyright 2023 Lance P. Rhodes, Ph.D. All rights reserved.

Grade Breakdown	
A .cpp file containing code for a linked-list queue has been submitted.	N/A
The `Node` struct has been coded with two data members (`data`, type `string` and `next`, type `Node` pointer).	5 points
The `Queue` class has been coded with 3 data members, one constructor (or default), one destructor, and six member functions.	10 points
The `~Queue` destructor correctly removes all nodes created on the heap; this may be done with repeated calls to the `dequeue()` function.	5 points
The `enqueue()` function correctly adds a new data node to the end of the queue.	30 points
The `dequeue()` function correctly removes a data node from the front of the queue.	15 points
The `peek()` function returns the data string located at the front of the queue.	5 points
The `isEmpty()` function correctly returns `true` if the queue contains no nodes; `false` if the queue contains at least one node.	5 points
The `size()` function correctly returns the number of nodes in the queue. (Which further entails correctly adding to and subtracting from the `count` variable).	10 points
The `print()` function appropriately walks the queue, displaying every node from front to back in some decorated format; the `print()` function displays a message if the queue is empty.	5 points
The provided test code has been implemented and displays the test values correctly.	5 points
At least three additional tests have been provided (enqueue, dequeue, peek, size, etc.)	5 points
Total	**100 points**

© Copyright 2023 Lance P. Rhodes, Ph.D. All rights reserved.